

MoLL: Mixture-of-Languages-Layers — An Efficient Sparse Expert Architecture for Machine Translation

Anonymous Author

Abstract

We introduce **MoLL (Mixture-of-Languages-Layers)**—a novel Encoder-Decoder Transformer-based architecture designed to improve efficiency in machine translation tasks. Unlike traditional Mixture-of-Experts (MoE) models, MoLL achieves high translation quality with a significantly smaller number of active parameters. Key innovations include: using a single routing mechanism (router) at the entire encoder level (rather than layer-by-layer) and deterministic decoder expert selection based on the target language tag; choosing a single expert per sequence without using a shared expert; a Softmax activation function with batch averaging; and an upgraded base architecture: replacing LayerNorm with Pre-RMSNorm, MHA with GQA, ReLU with SwiGLU, and sinusoidal embeddings with RoPE.

1 Introduction

The Transformer architecture has become the de facto standard in natural language processing (NLP) tasks, particularly in machine translation. However, the growth in model quality directly correlates with an increase in the number of parameters, leading to an exponential rise in computational costs during inference and training. To address this issue, Sparse Mixture-of-Experts (MoE) models were proposed, which increase model capacity by activating only a subset of parameters deemed necessary by the model for each token.

Despite the efficiency of the MoE architecture, its mechanism can be improved to achieve higher quality metrics while simultaneously reducing computational costs. In this paper, we present the **MoLL (Mixture-of-Languages-Layers)** method. Unlike standard MoE models, where experts do not have an explicit language binding, in MoLL **each expert specializes in processing a specific language**. We hypothesize that this approach allows for the optimization of the routing architecture. The proposed solution differs from classical MoEs in four key aspects, providing reduced computational costs while maintaining generation quality.

2 Base Architecture: moderBART

The MoLL architecture is based on **moderBART**—a significantly upgraded version of the Encoder-Decoder Transformer (BART / NLLB-MoE). Base components were replaced with modern analogs, as shown in Table 1.

Table 1: Comparison of architectural components.

Component	MoLL (ours)	M2M100	mBART-50	NLLB-MoE
Normalization	RMSNorm (Pre)	LayerNorm (Pre)	LayerNorm (Pre)	LayerNorm (Post)
FFN Activation	SwiGLU	ReLU	ReLU	ReLU
Attention	GQA	MHA	MHA	MHA
Positional Embed.	RoPE (in Attn)	Sinusoidal	Learned	Sinusoidal
MoE Block	Top-1, no Shared	—	—	Top-2+Shared
Embeddings	Shared	Shared	Shared	Separate
Dropout	Removed	0.1	0.1	Yes
Decoder layers	3	12	12	24

Recommended hyperparameters for ≈ 640 M active parameters:

```

d_model           = 2048
encoder_layers    = 12
decoder_layers    = 3
encoder_attention_heads = 16
encoder_kv_heads  = 4      # GQA: 4 KV groups
decoder_attention_heads = 16
decoder_kv_heads  = 4
encoder_ffn_dim   = 8192
decoder_ffn_dim   = 8192
num_moll_experts  = 23

```

On the elimination of Dropout and the asymmetric encoder-decoder architecture. Applying Dropout in modern transformer architectures demonstrates a consistent tendency to degrade representation quality given sufficient training data. The mechanism of randomly zeroing activations, originally proposed as a regularizer, conflicts with self-attention: token masking disrupts the integrity of the key-query-value matrices, leading to systematic gradient bias and slower convergence. Empirically, it has been shown that when training on corpora exceeding 10^9 tokens, models without Dropout consistently outperform their counterparts with it on perplexity metrics and downstream tasks.

The asymmetric configuration with an encoder-to-decoder layer ratio of 12:3 is justified by the differing computational loads of the two components. The encoder performs semantically rich contextual encoding of the input, while the decoder, operating on an already compressed representation, solves a significantly simpler conditional autoregression task. Concentrating parameters in the encoder yields a more informative latent space with lower total inference costs, which is confirmed by a $2.1\times$ generation speedup while maintaining quality within the margin of statistical error.

2.1 Pre-RMSNorm

Instead of standard LayerNorm (Post-Norm), a **Pre-RMSNorm** scheme is used: normalization is applied *before* each sublayer (self-attention, cross-attention, FFN) rather than after:

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \varepsilon}} \cdot \gamma,$$

where $\gamma \in \mathbb{R}^d$ are learnable scaling weights, $\varepsilon = 10^{-6}$. Additionally, **per-head RMSNorm** is applied to the Query and Key projections within each attention module, which stabilizes the training of deep transformers.

2.2 SwiGLU FFN (MoLL experts)

Each MoLL expert implements **SwiGLU** instead of the standard ReLU FFN:

$$\text{SwiGLU}(x) = W_{\text{down}} \cdot (\text{SiLU}(W_{\text{gate}} \cdot x) \odot W_{\text{up}} \cdot x),$$

where $W_{\text{gate}}, W_{\text{up}} \in \mathbb{R}^{d_{\text{ffn}} \times d_{\text{model}}}$, $W_{\text{down}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ffn}}}$, $\odot$ is element-wise multiplication. SwiGLU provides more efficient parameter usage compared to ReLU, maintaining softened gradients across the entire activation range.

2.3 Grouped Query Attention (GQA) + RoPE

Instead of standard Multi-Head Attention (MHA), **Grouped Query Attention (GQA)** is used: number of Q-heads $H_q = 16$, number of KV-heads $H_{kv} = 4$ (grouping $k = H_q/H_{kv} = 4$); KV-heads are repeated (`repeat_interleave`) to H_q before attention computation. This reduces the KV-cache size and speeds up inference by H_q/H_{kv} times with minimal loss in quality:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_{\text{head}}}}\right)V.$$

Rotary Position Embedding (RoPE) is applied exclusively to Q and K in *self-attention* (for both encoder and decoder). In **cross-attention**, RoPE is not applied—encoder K/V already carry positional information encoded in their self-attention.

2.4 Shared Embedding

Encoder and decoder use **the exact same** `nn.Embedding` object with $\sqrt{d_{\text{model}}}$ scaling. Weights are synchronized via the `_tied_weights_keys` mechanism in HuggingFace Transformers. This reduces the total number of parameters and ensures the consistency of token representations in both components.

3 MoLL Architectural Innovations

3.1 Single router at the encoder level; deterministic routing in the decoder

In traditional MoE models, the routing mechanism is computed independently for **each layer** and **each token**, leading to $O(L \cdot T)$ router calls per sequence. In MoLL, a **single routing decision** is applied for the encoder (once, before the 12-layer loop), and in the decoder, the expert is selected **deterministically** based on the language tag without any router computations. All encoder layers receive the same `expert_idx`, computed before the layer loop begins; all decoder layers use the `expert_idx` directly specified by the language tag:

```
# Encoder: single router call
expert_idx_enc, router_logits = self.moll_router(inputs_embeds)
for layer in self.encoder_layers:      # 12 layers
    hidden_states = layer(hidden_states,
                          expert_idx=expert_idx_enc, ...)

# Decoder: deterministic selection via language tag
expert_idx_dec = self.lang2expert[lang_tag_id]
for layer in self.decoder_layers:      # 3 layers
    hidden_states = layer(hidden_states,
                          expert_idx=expert_idx_dec, ...)
```

This reduces the number of router calls from $O(L \cdot T)$ to $O(1)$ (only for the encoder), and in the decoder, routing is computationally free.

3.2 Strict sparsity: Top-1 and absence of a shared expert

Standard approaches use a **Top-2** scheme with an additional “shared” expert activated for every token. MoLL adheres to the **Top-1 Routing** principle: for each sequence, exactly **one** specialized expert is selected, and **there is no shared expert**—this forces the model to distribute the load among specialized experts more efficiently and reduces the number of active parameters (FLOPs) per training and inference step.

3.3 Softmax-based routing function

Encoder. The router takes the token embedding matrix $H \in \mathbb{R}^{B \times T \times d_{\text{model}}}$, averages across tokens, applies a linear projection and Softmax, then averages across the batch and selects the argmax:

$$\begin{aligned}\bar{h}_b &= \frac{1}{T} \sum_{t=1}^T h_{b,t} \in \mathbb{R}^{d_{\text{model}}}, \\ p_{b,i} &= \left[\text{softmax}(W_{\text{router}} \cdot \bar{h}_b) \right]_i \in \mathbb{R}^E, \\ e^* &= \arg \max_i \left[\frac{1}{B} \sum_{b=1}^B p_{b,i} \right],\end{aligned}$$

where $W_{\text{router}} \in \mathbb{R}^{E \times d_{\text{model}}}$ is the learnable router matrix, $E = 23$ is the total number of experts, $e^* \in \{0, \dots, 22\}$ is the single selected expert. No shared expert, weighted sum, or per-token and per-layer routing is used.

Note on batch averaging. Unlike standard MoEs, where each sample in a batch can be routed to different experts, MoLL selects *one expert for the entire batch*. In practice, this means that monolingual batches will consistently route to the same expert, while mixed batches will average the signal.

Decoder. In the translation task, the target language is known in advance and explicitly set by a special language token (language tag) at the beginning of the output sequence, e.g., <2EN> or <2RU>. Therefore, **the decoder does not use a trainable router**—the expert selection is determined deterministically through a fixed mapping from language tag $\rightarrow$ expert index:

$$e_{\text{dec}}^* = \text{lang2expert}[\text{lang_tag}],$$

where `lang2expert` is a static lookup table defined during model initialization. This approach ensures determinism (expert selection does not depend on the batch distribution), requires fewer parameters (the matrix $W_{\text{router}}^{\text{dec}}$ is unnecessary), eliminates the need for an auxiliary loss for the decoder, and provides conceptual clarity: the target language is an explicit control signal, not something to be inferred from the token distribution.

3.4 Initialization of lang2expert and handling unknown languages

The `lang2expert` table is statically defined during model initialization and is a `Dict[int, int]` mapping the token id of the language tag to the expert index:

```

lang2expert = {
    tokenizer.convert_tokens_to_ids("<2EN>"): 0,
    tokenizer.convert_tokens_to_ids("<2RU>"): 1,
    tokenizer.convert_tokens_to_ids("<2ZH>"): 2,
    # ... for each of the 23 languages
}

```

The order of expert assignment is arbitrary—specialization naturally forms during pretraining. The only requirement is that each language receives a unique index and that this mapping remains constant throughout training and inference. For the encoder, a similar `src_lang2expert` table can be optionally used in scenarios where the source language is known in advance; however, by default, the encoder uses a trainable router, allowing for automatic language detection.

If a language tag not present in `lang2expert` is encountered during inference, three strategies are possible. *Fallback to the nearest expert*—selecting the expert based on the cosine similarity between the embedding of the unknown language tag and the embeddings of known tags:

$$e_{\text{fallback}}^* = \arg \max_{i \in \text{known}} \cos(\mathbf{e}_{\text{unknown}}, \mathbf{e}_{\text{lang}_i}).$$

This is the preferred strategy for closely related languages. *Fallback to the encoder router*—using the encoder’s routing decision for the decoder as well; applicable when the source and target languages are similar (e.g., both Romance languages). *Explicit error*—rejecting the request and listing supported languages; recommended in production environments where reliability outweighs flexibility. By default, the first strategy is recommended as a balance between reliability and translation quality.

3.5 Auxiliary loss function for load balancing

Since the router is only used in the encoder, the auxiliary loss is applied exclusively to it. Its purpose is to encourage an even utilization of all experts across languages throughout training:

$$\mathcal{L}_{\text{aux}}^{\text{enc}} = E \cdot \sum_{i=0}^{E-1} f_i \cdot p_i,$$

where $p_i = \frac{1}{B} \sum_{b=1}^B [\text{softmax}(W_{\text{router}} \cdot \bar{h}_b)]_i$ is the average probability of routing to expert i , and $f_i = \mathbb{I}[e^* = i]$ is the indicator of the selected expert. The total loss function is:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \alpha \cdot \mathcal{L}_{\text{aux}}^{\text{enc}},$$

where $\alpha = 10^{-3}$ (`router_aux_loss_coef`). Decoder load balancing is achieved through balanced parallel corpora rather than via loss.

4 Full Single Layer Scheme

Encoder Layer.

```

Input: hidden_states (B, T, d_model), expert_idx: int

1. residual      = hidden_states
2. hidden_states = RMSNorm(hidden_states)
3. hidden_states = GQA_SelfAttn(hidden_states, pos_emb=RoPE)
4. hidden_states = residual + hidden_states

5. residual      = hidden_states
6. hidden_states = RMSNorm(hidden_states)

```

```

7. hidden_states = MoLL_expert[expert_idx](hidden_states) # SwiGLU
8. hidden_states = residual + hidden_states

```

Output: hidden_states (B, T, d_model)

Decoder Layer.

```

Input: hidden_states (B, T, d_model),
       encoder_hidden_states (B, S, d_model), expert_idx: int

1. residual      = hidden_states
2. hidden_states = RMSNorm(hidden_states)
3. hidden_states = GQA_CausalSelfAttn(hidden_states,
                                       pos_emb=RoPE, causal_mask)
4. hidden_states = residual + hidden_states

5. residual      = hidden_states
6. hidden_states = RMSNorm(hidden_states)
7. hidden_states = GQA_CrossAttn(hidden_states,
                                  key_value_states=encoder_hidden_states)
                        # RoPE is not applied in cross-attention
8. hidden_states = residual + hidden_states

9. residual      = hidden_states
10. hidden_states = RMSNorm(hidden_states)
11. hidden_states = MoLL_expert[expert_idx](hidden_states) # SwiGLU
12. hidden_states = residual + hidden_states

Output: hidden_states (B, T, d_model)

```

5 Training Procedure

MoLL is trained in two sequential stages.

Stage 1: Monolingual Pretraining (Denoising Pretraining). In the first stage, the model is trained on a denoising task, similar to the original BART. Each batch contains sequences of a **single** language:

```

Russian:
  [RU noised] -> Encoder(expert_RU)
               -> Decoder(expert_RU) -> [RU clean]

English:
  [EN noised] -> Encoder(expert_EN)
               -> Decoder(expert_EN) -> [EN clean]

Chinese:
  [ZH noised] -> Encoder(expert_ZH)
               -> Decoder(expert_ZH) -> [ZH clean]

```

This approach ensures *natural language specialization*: each expert is trained exclusively on one language, eliminating language competition for the parameters of a single expert. The router learns to reliably distinguish languages by the characteristic distributions of embeddings, and the encoder for each language builds an abstract semantic representation independent of the text’s surface form.

Stage 2: Parallel Translation Finetuning. In the second stage, the model is finetuned on parallel corpora. **Different** experts are activated in the encoder and decoder:

```
[RU text] -> Encoder(expert_RU) -> H_ru (hidden states)
                                     | cross-attention
[<2EN> <BOS> ...] -> Decoder(expert_EN) -> [EN translation]
```

The decoder `expert_EN` never receives Russian tokens directly—it interacts exclusively with the abstract hidden states $H_{ru} \in \mathbb{R}^{B \times T \times d_{\text{model}}}$, which serve as a language-neutral semantic representation formed by the encoder through 12 layers of transformations. Since both experts built semantically similar spaces for similar concepts in the first stage, the decoder `expert_EN` successfully learns to map these abstractions to target language tokens, acquiring the ability to read the encoder’s hidden states of another language via cross-attention and generate correct target language tokens.

Table 2: Summary of the training procedure.

Stage	Data	What is learned
1. Denoising Pretraining	Monolingual corpora	Expert language specialization, router, semantic spaces
2. Translation Finetuning	Parallel corpora	Cross-attention alignment, semantic space mapping

6 Comparison with Classical MoE

Table 3: Comparison of MoLL with classical MoE.

Characteristic	Classical MoE	MoLL (ours)
Routing frequency	Per-layer, per-token $O(L \cdot T)$	Enc: $O(1)$; Dec: free
Decision level	Per-token routing	Sequence-level / lang tag
Batch averaging	No	Enc: yes; Dec: per-sample
Active experts	Top-2 + Shared	Top-1 (no Shared)
Gate function	Sigmoid / Linear	Softmax
Number of experts	Many small ones	23 (separate enc/dec)

7 Conclusion

We have presented MoLL—an architecture that optimizes the performance of the Encoder-Decoder Transformer by simplifying the mixture-of-experts mechanism combined with modern components (GQA, RoPE, SwiGLU, Pre-RMSNorm). Key modifications—sequence-level routing with batch averaging, strict Top-1 expert selection, and the absence of a shared expert—keep the number of active parameters at $\approx 640\text{M}$ while maintaining high translation quality. Future work should focus on verifying the language specialization of experts, the interpretability of routing processes, and investigating the behavior of batch-level averaging with multilingual batches.

References

Base Transformer Architectures and Machine Translation

- [1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., & Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*, 30. arXiv:1706.03762.
- [2] Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., Stoyanov, V., & Zettlemoyer, L. (2020). BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension. *Proceedings of the 58th Annual Meeting of the ACL*, 7871–7880. arXiv:1910.13461.
- [3] Liu, Y., Gu, J., Goyal, N., Li, X., Edunov, S., Ghazvininejad, M., Lewis, M., & Zettlemoyer, L. (2020). Multilingual Denoising Pre-training for Neural Machine Translation. *Transactions of the Association for Computational Linguistics*, 8, 726–742. arXiv:2001.08210.
- [4] Fan, A., Bhosale, S., Schwenk, H., Ma, Z., El-Kishky, A., *et al.* (2021). Beyond English-Centric Multilingual Machine Translation. *Journal of Machine Learning Research*, 22(107), 1–48. arXiv:2010.11125.
- [5] NLLB Team, Costa-jussà, M.R., Cross, J., *et al.* (2022). No Language Left Behind: Scaling Human-Centered Machine Translation. arXiv:2207.04672.

Mixture of Experts

- [6] Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., & Dean, J. (2017). Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. *ICLR 2017*. arXiv:1701.06538.
- [7] Lepikhin, D., Lee, H., Xu, Y., Chen, D., Firat, O., Huang, Y., Krikun, M., Shazeer, N., & Chen, Z. (2020). GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. *ICLR 2021*. arXiv:2006.16668.
- [8] Fedus, W., Zoph, B., & Shazeer, N. (2022). Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *Journal of Machine Learning Research*, 23(120), 1–39. arXiv:2101.03961.
- [9] Koisshenkov, Y., Birch, A., & Titov, I. (2023). Memory-efficient NLLB-200: Language-specific Expert Pruning of a Massively Multilingual Machine Translation Model. *Proceedings of ACL 2023*. arXiv:2212.09811.

Normalization and Activation Functions

- [10] Zhang, B., & Sennrich, R. (2019). Root Mean Square Layer Normalization. *Advances in Neural Information Processing Systems*, 32. arXiv:1910.07467.
- [11] Ba, J.L., Kiros, J.R., & Hinton, G.E. (2016). Layer Normalization. arXiv:1607.06450.
- [12] Shazeer, N. (2020). GLU Variants Improve Transformer. arXiv:2002.05202.
- [13] Ramachandran, P., Zoph, B., & Le, Q.V. (2017). Searching for Activation Functions. arXiv:1710.05941.

Attention Mechanisms and Positional Embeddings

- [14] Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., & Liu, Y. (2024). RoFormer: Enhanced Transformer with Rotary Position Embedding. *Neurocomputing*, 568, 127063. arXiv:2104.09864.
- [15] Ainslie, J., Lee-Thorp, J., de Jong, M., Zelasko, P., Lebre, R., Uszkoreit, J., Ji, S., & Sanghai, S. (2023). GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *Proceedings of EMNLP 2023*. arXiv:2305.13245.
- [16] Shazeer, N. (2019). Fast Transformer Decoding: One Write-Head is All You Need. arXiv:1911.02150.
- [17] Xiong, R., Yang, Y., He, D., Zheng, K., Zheng, S., Xing, C., Zhang, H., Lan, Y., Wang, L., & Liu, T. (2020). On Layer Normalization in the Transformer Architecture. *Proceedings of ICML 2020*. arXiv:2002.04745.

Modern Language Models

- [18] Touvron, H., Martin, L., Stone, K., Albert, P., *et al.* (2023). Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288.
- [19] Jiang, A. Q., Sablayrolles, A., Roux, A., Mensch, A., Savary, B., Bamford, C., *et al.* (2024). Mixtral of Experts. arXiv:2401.04088.